

# Grid & Chart API Contract Guide

What services send for every grid and chart, what each field means, every cell style with the JSON that produces it, and how the UI turns it all into grids, heatmaps and charts.

**Audience:** UI, services, QA   **Covers:** Grid Config · Grid Data · Chart Config   **Version:** Oct 2026

## 1. The idea in one paragraph

Every grid is described by two responses. The **Grid Config** says what the grid is: its title, columns, how each column looks and behaves, which rows are totals, and which rows are pinned. The **Grid Data** carries only the rows (**rowData**). The UI merges them back into one complete Grid Response v2. A chart is **not** a separate data source: it is drawn from that same merged grid response. A third response, the **Chart Config**, supplies only what a grid can't express (chart type, stacking, axes, legend, series colours).

### PRINCIPLE 1

#### Backend decides WHAT

Columns, row roles, statuses and totals come from services. A new column needs no UI release.

### PRINCIPLE 2

#### UI owns HOW

Fonts, widths, palettes, tooltips and animation live in the UI. No functions are ever sent.

### PRINCIPLE 3

#### Tones, not colours

Services send meaning (*negative, warning, series1*). Raw hex only as a deliberate override.

## 2. The three responses

### RESPONSE A Grid Config

Grid Response v2 **minus rowData**. Requested with the **same params** as the data, because totals and pinned rows follow the filters.

### RESPONSE B Grid Data

Only **{ rowData }**: plain rows, one key per column field, plus a stable **id**. No styling, no row roles.

### RESPONSE C Chart Config

One per dashboard: chart settings keyed by **chartConfigId**. Holds no data, only how to draw a grid response as a chart.

## 3. How a grid or chart loads

This flow runs for every grid on the page, and all grids run it in parallel. With 10–20 grids, the page still loads in roughly the time of the slowest single grid.

1

**Build the params**

The UI collects current filter values (COB, PCOB, Compare, Region, Variant, Attribute, Display, Days) and any selection the grid depends on.



2

**Request Grid Config and Grid Data together**

Both go out at the same moment with **identical** params. The card shows its spinner until both arrive.



3

**Validate and merge**

Tolerant validation: unknown keys are kept and missing optional blocks default to empty. Then **config + sc-camel-row-data = Grid Response v2**. A row that is also pinned is shown only once.



4

**Render a grid — or transform it into a chart**

Grid view renders the merged response directly. Chart view passes the same response with its Chart Config to the chart builder. Switching Grid ⇌ Chart makes **no new request**.



5

**React to change**

When a filter or selection the grid depends on changes, steps 1–4 run again. Every params combination already seen is cached, so going back is instant.

**DRILLDOWN · STANDALONE GRID**

A row or cell click sends that row's **id** as *rowId* to the trend, Component and FS type grids and charts, which refetch. Clicking the row again or pressing Esc sends **all**.

**DRILLDOWN · LIMITS GRID**

The **UI selects row 1** on load, so the limit chart always has data. A click picks another limit. If a filter removes the selected row, the UI falls back to row 1.

## 4. Grid Config — the blocks

Only **schemaVersion**, **gridId** and **columnDefs** are required. Every other block may be omitted and is treated as empty.

```
{
  "schemaVersion": "2.0",           // required
  "gridId": "standalone-var-svar",  // required, stable
  "gridMetaData": { ... },          // grid-level options
  "columnDefs": [ ... ],            // required — which columns
  "columnMetadata": { "field": { ... } }, // how each column looks
  "rowMetadata": { "rowId": { ... } },  // row roles + flags
  "cellMetadata": { "rowId": { "field": { ... } } },
  "pinnedTopRows": [ ... ],
  "pinnedBottomRows": [ ... ]       // Grand Total, limit row
}
```

### 4.1 gridMetaData

| FIELD                                        | WHAT IT DOES                                                                                                                                                             |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>title · subtitle</code>                | Card heading and sub-line. Tokens such as <i>{filter.cob}</i> are filled in by the UI.                                                                                   |
| <code>defaultColDef</code>                   | Defaults for all columns: sortable, filter, resizable, floatingFilter.                                                                                                   |
| <code>rowSelection</code>                    | <i>none</i>   <i>single</i> . Whether a row can be selected.                                                                                                             |
| <code>getRowId</code>                        | Name of the field holding the row key. Always <i>"id"</i> .                                                                                                              |
| <code>pagination · paginationPageSize</code> | Off for all current grids (client-side rows, auto height).                                                                                                               |
| <code>masterDetail · pivotMode</code>        | Reserved; <i>false</i> today.                                                                                                                                            |
| <code>valueUnit</code>                       | Unit shown on screen and the multiplier back to the actual value, e.g. <i>{"label": "\$MM", "multiplier": 1000000}</i> . The plain hover tooltip shows the actual value. |
| <code>heatmap</code>                         | Grid-level heatmap over <b>all numeric columns</b> ; see 6.5. <i>null</i> = no heatmap.                                                                                  |

### 4.2 columnDefs

Which columns exist and in what order. A column with **children** is a group header (e.g. "Monte Carlo (VaR)" over COB / PCOB / Diff). Widths are owned by the UI: unpinned columns flex to fill the card, and a sent *width* acts only as a minimum.

| FIELD                                                               | MEANING                                                                                               |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>headerName</code>                                             | Header text. Always write <b>VaR</b> , never VAR (SVaR, CVaR, MC VaR...).                             |
| <code>field</code>                                                  | Key in each row of <code>rowData</code> . Must be unique in the grid.                                 |
| <code>colType</code>                                                | <i>StringType</i> (left-aligned) · <i>DoubleType</i> (right-aligned, monospace) · <i>CustomType</i> . |
| <code>pinned</code>                                                 | <i>left</i> keeps the column visible while scrolling sideways.                                        |
| <code>hide</code> · <code>headerTooltip</code> · <code>width</code> | Hidden but still in the data · header hover text · minimum width (optional).                          |
| <code>children</code>                                               | Nested columns under a group header.                                                                  |

```
"columnDefs": [
  { "headerName": "Volcker Desk Name", "field": "name", "pinned": "left", "colType": "StringType" },
  { "headerName": "Monte Carlo (VaR)", "children": [
    { "headerName": "COB", "field": "mcVarCob", "colType": "DoubleType" },
    { "headerName": "PCOB", "field": "mcVarPcob", "colType": "DoubleType" },
    { "headerName": "Diff", "field": "mcVarDiff", "colType": "DoubleType" } ] }
]
```

### 4.3 rowMetadata · pinned rows · cellMetadata

| FIELD                                                      | MEANING                                                                                                                                                        |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rowMetadata[id].rowType</code>                       | <i>GRAND_TOTAL</i> · <i>SUB_TOTAL</i> · <i>LIMIT</i> . Drives total styling, chart row picking and heatmap exclusion.                                          |
| <code>bold</code> · <code>bgColor</code>                   | Force bold text / a row background (hex override).                                                                                                             |
| <code>excludeHeatMap</code>                                | This row is not shaded and not counted in the heatmap range.                                                                                                   |
| <code>selectable</code>                                    | <i>false</i> = clicking this row does nothing.                                                                                                                 |
| <code>status</code>                                        | <i>BREACH</i> · <i>WARNING</i> on limit rows; drives the alert strip.                                                                                          |
| <code>error</code> · <code>actions</code>                  | Per-row error message · per-row action switches, e.g. <code>{"edit":{"enabled":false}}</code> .                                                                |
| <code>pinnedTopRows</code> / <code>pinnedBottomRows</code> | Full rows (same fields as <code>rowData</code> ) that stay fixed above / below the scroll area. Give them a <code>rowType</code> in <code>rowMetadata</code> . |
| <code>cellMetadata[id][field]</code>                       | One-cell overrides: <code>bgColor</code> , <code>color</code> , <code>tooltip</code> , <code>icon</code> . See 6.9.                                            |

## 5. Grid Data

Only one key. Each row has an **id** plus one key per column field. Values are raw numbers in the unit the grid displays (e.g. \$MM); the UI formats them. No `rowType`, `status`, colours or labels go in rows — those belong in the config.

```
{
  "rowData": [
    { "id": "row_1", "name": "Equity Derivatives NA", "region": "NA",
      "mcVarCob": 28.17, "mcVarPcob": 26.5, "mcVarDiff": 1.67 }
  ]
}
```

**Rules:** ids are stable across requests, so selection survives a refresh · *null* means "no value" and is shown as a dash · a field present in rowData but not in columnDefs is ignored.

## 6. Cell styles — every type with its JSON

A cell's look is chosen by `columnMetadata[field].type`, refined by **format**, **tone** and the row's **rowMetadata**. Below, each style shows what the user sees and the exact config that produces it.

### 6.1 Text — names, ids, regions. Left-aligned.

Equity Derivatives NA

```
"name": { "type": "text" }
```

### 6.2 Value — plain number; optional muted or sign tone.

1,904.65 default

26.50 muted

+3.20 sign

```
"exposure": { "type": "value",  
  "format": { "style": "fixed", "decimals": 2 } },  
"mcVarPcob": { "type": "value", "tone": "muted" },  
"chg1d": { "type": "value", "tone": "sign",  
  "format": { "decimals": 2, "showPlus": true } }
```

### 6.3 Magnitude bar — column-level heat: tinted bar sized to the column max. Value is **bold**.

37.42

18.10

```
"mcVarCob": {  
  "type": "bar",  
  "bar": { "mode": "magnitude" },  
  "format": { "style": "fixed", "decimals": 2 }  
}
```

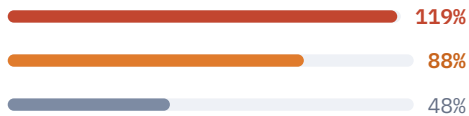
### 6.4 Diverging bar — change columns: mini bar left / right of zero.

+3.49

-0.85

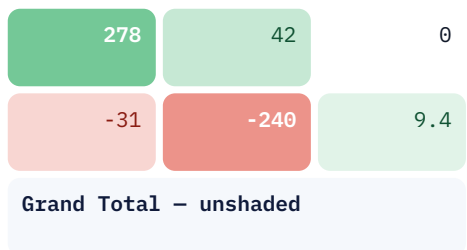
```
"mcVarDiff": {  
  "type": "bar",  
  "bar": { "mode": "diverging" },  
  "tone": "sign",  
  "format": { "decimals": 2, "showPlus": true }  
}
```

#### 6.5 Progress bar — utilisation vs a cap, coloured by thresholds.



```
"utilization": {
  "type": "bar",
  "bar": { "mode": "progress", "max": 1.2,
    "thresholds": [ { "gte": 0.8, "tone": "warning" },
      { "gte": 1.0, "tone": "negative" } ] },
  "format": { "style": "percent", "decimals": 0 },
  "sort": "desc"
}
```

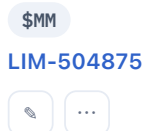
#### 6.6 Grid-level heatmap — one colour scale across every numeric cell.



```
"gridMetadata": {
  "heatmap": { "scope": "grid", "centerline": 0,
    "min": null, "max": null,
    "positiveColor": null, "negativeColor": null }
},
"rowMetadata": {
  "grand_total": { "rowType": "GRAND_TOTAL",
    "excludeHeatMap": true } }
```

Covers **all numeric columns**; no field list is sent. Shade strength =  $\sqrt{(|\text{value} - \text{centerline}| / \text{grid max})}$ . Rows opt out with **excludeHeatMap**. Zero (or a value that displays as zero) is never coloured. *min/max null* = UI computes the range; *colour null* = UI palette.

#### 6.7 Badge · Link · Action — small chips, clickable text, row buttons.



```
"unit": { "type": "badge",
  "badge": { "static": true, "value": "$MM" } },
"limitId": { "type": "link",
  "link": { "handler": "openLimitDetail" } },
"actions": { "type": "action",
  "action": { "items": [ { "id": "edit", "icon": "edit",
    "handler": "onRowAction" } ] } }
```

Handlers are **names**, resolved in the UI registry — functions are never sent.

### 6.8 Row styles — Grand Total, Sub Total, limit row, selected row.

#### Selected row

AMER Total SUB\_TOTAL · 700

Grand Total 800

EQ T2 Limit · -500|500 LIMIT

```
"rowMetadata": {
  "amer_total": { "rowType": "SUB_TOTAL", "bold": true },
  "grand_total": { "rowType": "GRAND_TOTAL", "bold": true,
    "excludeHeatMap": true, "selectable": false },
  "eq_t2_limit": { "rowType": "LIMIT",
    "excludeHeatMap": true, "selectable": false },
},
"pinnedBottomRows": [
  { "id": "grand_total", "region": "Grand Total", "delta": 25 },
  { "id": "eq_t2_limit", "region": "EQ T2 Limit", "delta": "-500|500" }
]
```

Limit values are sent as **"min|max"** strings (" " = no limit). The UI shows them bold, upright, left-aligned. The selected-row highlight is UI state and is never sent.

### 6.9 Cell overrides — one cell gets its own colour, icon or note.

12.40

```
"cellMetadata": {
  "row_4": { "mcVarDiff": { "bgColor": "#ffff3d6", "color": "#000000",
    "icon": "flag", "tooltip": "Unusual move: +38% v" } }
}
```

**Colour order (last wins):** UI tone → column config hex → rowMetadata.bgColor → cellMetadata.

### 6.10 Number format & negatives

| format                                | 1234.567                      | -12.3   |
|---------------------------------------|-------------------------------|---------|
| { "style": "fixed", "decimals": 2 }   | 1,234.57                      | -12.30  |
| { "style": "compact" }                | 1,235                         | -12.3   |
| { "style": "percent", "decimals": 0 } | 123,457%                      | -1,230% |
| ... "showPlus": true                  | +1,234.57                     | -12.30  |
| ... "empty": ""                       | null shows blank instead of — |         |

Negative display is an **application setting**: SPARC shows **-123 in red** everywhere, even if a service sends *"negative"*: *"parentheses"*. Other apps may choose parentheses.

## 6.11 Tooltips — plain (actual value) and the dark detail card.

28,170,000

### VaR EXPOSURE | DATA

| VaR Type | VaR   |
|----------|-------|
| COB      | 28.17 |
| Chg      | +1.67 |

```
"mcVarCob": { ...,  
  "tooltip": {  
    "renderer": "cellInfoTooltip",  
    "kicker": "VaR EXPOSURE | DATA",  
    "facts": [ { "label": "VaR Type", "value": "VaR" },  
               { "label": "Variant", "value": "{{filterLabel}}"  
    ],  
    "values": [ { "label": "COB", "field": "mcVarCob" },  
                 { "label": "Chg", "field": "mcVarDiff",  
                   "format": { "showPlus": true } } ],  
    "showShareOfTotal": false, "showHeatPct": false } }
```

Without a tooltip config, the **plain tooltip** shows the actual value (display value  $\times$  *valueUnit.multiplier*). Heatmap grids can add *showShareOfTotal* and *showHeatPct*. All tooltips share one hover delay.

## 7. Chart Config — turning a grid into a chart

A widget with **chartConfigId** can be shown as a chart. The chart builder reads the merged grid response and picks rows and fields according to the config. No chart titles are drawn (the card title is used), and charts fill the card height, with *height* as the minimum.

### timeSeries

One row per date (*categoryField*), one series per value field. Used for every VaR trend.

### fieldsAsCategories

Columns become x-axis categories, rows become series (e.g. regions stacked per Greek).

### rowsAsCategories

Each row is a category (top names as horizontal bars), each value field a series.

| FIELD                                                      | MEANING                                                                                                                 |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>type · stacking</code>                               | <i>line · area · column · bar · normal or percent.</i>                                                                  |
| <code>orientation</code>                                   | One of the three shapes above.                                                                                          |
| <code>categoryField · seriesField</code>                   | Field for the x-axis (timeSeries) / field naming each series (fieldsAsCategories).                                      |
| <code>valueFields · excludeFields</code>                   | Explicit list, or <i>""</i> = every visible DoubleType column. Fields not in the response are skipped (Display filter). |
| <code>includeRowTypes · excludeRowTypes</code>             | Pick rows by rowMetadata.rowType, e.g. only SUB_TOTAL rows, or never GRAND_TOTAL.                                       |
| <code>seriesLabels · seriesTones · seriesStyle</code>      | Legend names, tone per series (or a tone list), dash style.                                                             |
| <code>xAxis · yAxis · legend · stackTotals</code>          | Axis titles, label rotation, zero line, legend position/title, totals above stacks.                                     |
| <code>referenceLines · derivedSeries · colorByRatio</code> | Threshold line from a field · a ratio series on a secondary axis · bar colour by ratio thresholds (limit chart).        |
| <code>chartTooltip</code>                                  | Black table tooltip: grouping, row/column headers, name pattern, totals, footer.                                        |
| <code>height</code>                                        | Minimum height (px); the chart grows to fill its card.                                                                  |

```
// VaR trend line
"standalone-trend-line": {
  "type": "line",
  "orientation": "timeSeries",
  "categoryField": "cobDate",
  "valueFields": ["mcVar", "histVar",
                  "mcSvar", "histSvar"],
  "seriesLabels": { "mcVar": "MC VaR" },
  "seriesTones": { "mcVar": "series1" },
  "seriesStyle": { "histVar":
                  { "dash": "Dash" } },
  "legend": { "position": "top" },
  "height": 330
}
```

```
// Greek stacked by region
"region-greek-stacked": {
  "type": "column",
  "stacking": "normal",
  "orientation": "fieldsAsCategories",
  "seriesField": "region",
  "valueFields": "*",
  "excludeRowTypes": ["GRAND_TOTAL"],
  "seriesTones": { "AMER": "regionAmer",
                  "EMEA": "regionEmea" },
  "stackTotals": true,
  "legend": { "position": "bottom" },
  "height": 400
}
```

## 8. Errors, resilience and rules

### DATA FAILS

The card shows an error with **Retry**. Other cards keep working. The last good result stays visible during a refetch.

### CONFIG FAILS

Retried twice. Then the last good config for that grid is used (totals may be stale). If none is saved, the card shows Retry.

### CONTRACT VIOLATION

Wrong schemaVersion or a missing required block is **not retried**; the card explains what is wrong.

### TOLERANT READER

Unknown fields are ignored, never rejected, so services can add fields ahead of the UI.

### CHECKLIST FOR SERVICES

- ✓ Config + data accept identical params
- ✓ rowData holds raw values only
- ✓ Tones, not colours (hex only to override)
- ✓ "VaR" spelling in every label
- ✓ Every row has a stable **id**
- ✓ Totals pinned, typed in rowMetadata
- ✓ Handlers as names, never functions
- ✓ schemaVersion "2.x" on every config